



**RÉPUBLIQUE
FRANÇAISE**

*Liberté
Égalité
Fraternité*

ONERA

THE FRENCH AEROSPACE LAB

www.onera.fr

Filtrage bayésien et approximation particulière

TP ; filtre de Kalman

Frédéric Dambreville

frederic.dambreville@onera.fr

Informations

- Cours, cours/TP:
 - Frédéric Dambreville [frederic.dambreville@onera.fr] :
Filtre bayésien & Filtre de Kalman ; TP Kalman ; TP Bornes de Cramer Rao
 - Karim Dahia [karim.dahia@onera.fr] :
Filtrage particulière ; TP Fonctions d'importance ; TP Filtre particulière
-  **Rapports (4 TP) à rendre au plus tard 7 jours après chaque séance de TP**
 - Filtre de Kalman
 - Méthodes Monte Carlo, fonctions d'importance
 - Borne de Cramer Rao ; Optimisation de la trajectoire
 - Filtre particulière pour la navigation par corrélation de terrain
- Se mettre en binômes ; pas de langage imposé
- Théorie et preuves détaillées : <https://www.irisa.fr/aspi/legland/ensta/cours.pdf>

Plan

- Rappels
- TP1-Q0
- TP1-Q1
- TP1-Q2

Filtre de Kalman – récapitulatif

Initialisation $\leftarrow X_0 \sim \mathcal{N}(\hat{x}_0, P_0)$

Estimation (moyenne) : $\hat{x}_0$

Erreur (covariance) : P_0

Prédiction $\leftarrow p(x_k | y_{1:k-1}) = \int p(x_k | x_{k-1}) p(x_{k-1} | y_{1:k-1}) dx_{k-1}$

Estimation : $\hat{x}_{k|k-1} = F_k \hat{x}_{k-1} + c_k$

Erreur : $P_{k|k-1} = F_k P_{k-1} F_k^T + G_k Q_k G_k^T$

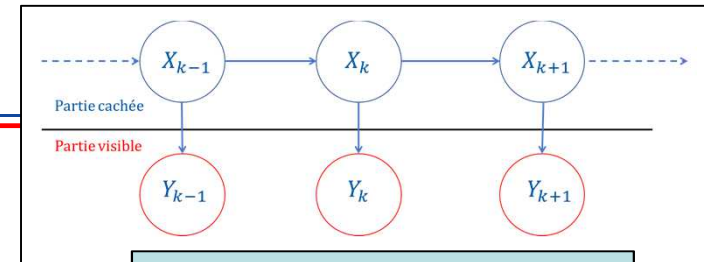
Correction $\leftarrow p(x_k | y_{1:k}) \propto p(y_k | x_k) p(x_k | y_{1:k-1})$

Gain de Kalman : $K_k = P_{k|k-1} H_k^T [H_k P_{k|k-1} H_k^T + R_k]^{-1}$

Terme d'innovation : $y_k - H_k \hat{x}_{k|k-1} - d_k$

Estimation : $\hat{x}_k = \hat{x}_{k|k-1} + K_k [y_k - H_k \hat{x}_{k|k-1} - d_k]$

Erreur : $P_k = [I - K_k H_k] P_{k|k-1}$



Linéaire Gaussien

...et même affine

Etat initial a priori

$X_0 \sim \mathcal{N}(\hat{x}_0, P_0)$

Modèle d'évolution

$X_k = F_k X_{k-1} + G_k \eta_k + c_k$ avec $\eta_k \sim \mathcal{N}(0, Q_k)$

Modèle d'observation

$Y_k = H_k X_k + v_k + d_k$ avec $v_k \sim \mathcal{N}(0, R_k)$

**Erreur déterministe
Indépendante de Y_k !**

Filtre de Kalman – illustration 2D : position 1D + vitesse 1D

Modèle d'évolution : $\Delta t = 1$

$$X_k = \begin{bmatrix} \text{Pos}_k \\ \text{Vel}_k \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} X_{k-1} + \eta_k \quad \text{avec} \quad \eta_k \sim \mathcal{N}\left(0, \begin{bmatrix} 1 & 0 \\ 0 & 0.001 \end{bmatrix}\right)$$

Modèle d'observation :

$$Y_k = \text{Pos}_k + v_k = [1 \quad 0] X_k + v_k \quad \text{avec} \quad v_k \sim \mathcal{N}(0, 1^2)$$

Initialisation :

Réel : $x_0 = \begin{bmatrix} 20 \\ 12 \end{bmatrix}$

Estimation : $\hat{x}_0 = \begin{bmatrix} 0 \\ 10 \end{bmatrix}$ et $P_0 = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}$

Prédiction : $t = 1$

Réel : $x_1 = \begin{bmatrix} 30.83 \\ 12.07 \end{bmatrix}$

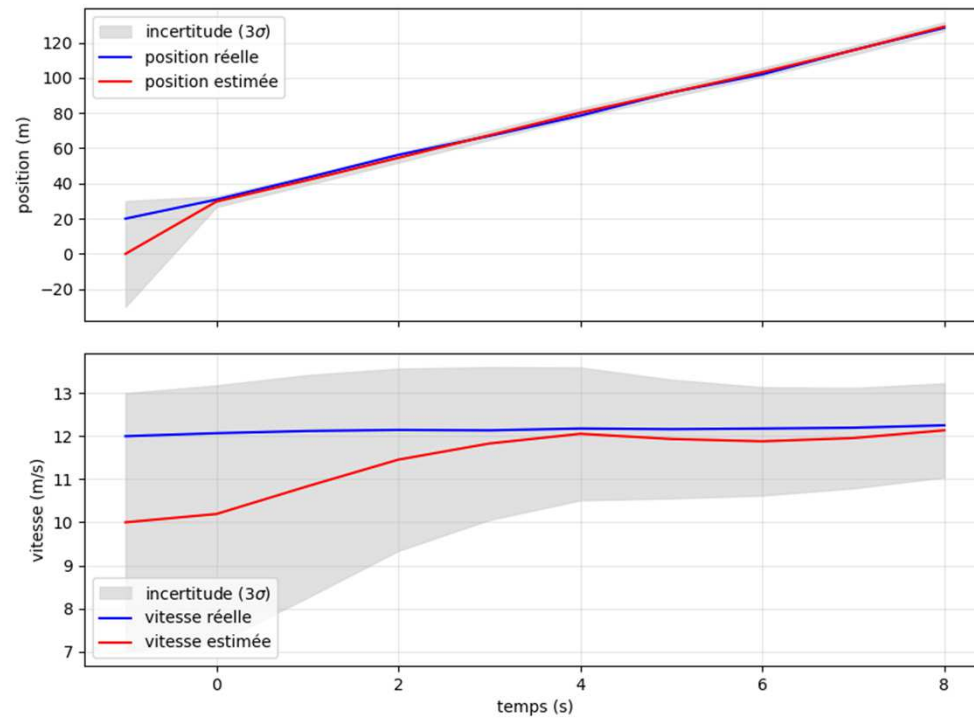
Correction : $t = 1$

Mesure : $y_1 = 29.91$

Estimation : $\hat{x}_{1|0} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$ et $P_{1|0} = \begin{bmatrix} 102 & 1 \\ 1 & 1.001 \end{bmatrix}$

Estimation : $K_1 = ???$; $\hat{x}_1 = \begin{bmatrix} 29.7 \\ 10.2 \end{bmatrix}$ et $P_1 = \begin{bmatrix} 0.9903 & 0.0097 \\ 0.0097 & 0.9913 \end{bmatrix}$

Filtre de Kalman – illustration 2D : position 1D + vitesse 1D



Forme de Joseph de l'erreur d'innovation

On peut remplacer : [forme simplifiée] $P_k = [I - K_k H_k] P_{k|k-1}$

→ 1 somme matricielle + 2 produits matriciels

par : [forme de Joseph] $P_k = [I - K_k H_k] P_{k|k-1} [I - K_k H_k]^T + K_k R_k K_k^T$

→ 2 sommes matricielle + 5 produits matriciels

- La forme de Joseph est plus couteuse, mais préserve mieux la **positivité** des matrices face aux erreurs numériques et évite des instabilités résultantes
- Souvent, la dimension m des mesures et la dimension n des états vérifient $m < n$, ce qui réduit le coût de calcul si on optimise l'ordre de calcul

Filtre de Kalman étendu – principe

Approximations affines locales:

$$f_k(x) = f_k(\hat{x}) + D_x f_k(\hat{x})(x - \hat{x}) + o(\|x - \hat{x}\|) \quad \text{et} \quad h_k(x) = h_k(\hat{x}) + D_x h_k(\hat{x})(x - \hat{x}) + o(\|x - \hat{x}\|)$$

On définit donc les approximations affines locales:

$$f_k(x) \simeq f_k(\hat{x}) + F_k(\hat{x})(x - \hat{x}) \quad \text{avec} \quad F_k(\hat{x}) = D_x f_k(\hat{x})$$
$$h_k(x) \simeq h_k(\hat{x}) + H_k(\hat{x})(x - \hat{x}) \quad \text{avec} \quad H_k(\hat{x}) = D_x h_k(\hat{x})$$

Initialisation inchangée

Estimation prédiction : $\hat{x}_{k|k-1} = f_k(\hat{x}_{k-1})$

Erreur prédiction : $P_{k|k-1} = F_k(\hat{x}_{k-1})P_{k-1}F_k^T(\hat{x}_{k-1}) + G_k(\hat{x}_{k-1})Q_kG_k^T(\hat{x}_{k-1})$

Gain de Kalman : $K_k = P_{k|k-1}H_k^T(\hat{x}_{k|k-1})[H_k(\hat{x}_{k|k-1})P_{k|k-1}H_k^T(\hat{x}_{k|k-1}) + R_k]^{-1}$

Terme d'innovation : $y_k - h_k(\hat{x}_{k|k-1})$

Estimation de correction : $\hat{x}_k = \hat{x}_{k|k-1} + K_k[y_k - h_k(\hat{x}_{k|k-1})]$

Erreur de correction : $P_k = [I - K_kH_k(\hat{x}_{k|k-1})]P_{k|k-1}$

Modèle d'évolution

$$X_k = f_k(X_{k-1}) + G_k(X_{k-1})\eta_k \quad \text{avec} \quad \eta_k \sim \mathcal{N}(0, Q_k)$$

Modèle d'observation

$$Y_k = h_k(X_k) + v_k \quad \text{avec} \quad v_k \sim \mathcal{N}(0, R_k)$$

Plan

- Rappels
- TP1-Q0
- TP1-Q1
- TP1-Q2

Partie Q0 – question a

Construire une classe « Filtre de Kalman »

Draft en Python: [kalman.py](#)

```
import numpy as np
class KalmanFilter:
    def __init__(self, x0, P0):
        self.x = np.asarray(x0, dtype=float)
        self.P = np.asarray(P0, dtype=float)
    def predict(self, F, Q):
        """
        Prediction:  $x_k|k-1 = F x_{k-1}$   $P_k|k-1 = F P_{k-1} F.T + Q$ 
        """
        # TODO
    def update(self, y, H, R):
        """
        Correction:  $innovation = y - H x$   $S = H P H.T + R$   $K = P H.T S^{-1}$ 
        """
        # TODO
        innovation = None
        S = None
        K = None
        return innovation, S, K
```

- ➔ La prédiction et la correction de l'estimation et de l'erreur sont placées dans les variables x0 et P0 de la classe
- ➔ On complètera la définition des 2 méthodes
- ➔ On testera la classe sur l'exemple du slide 6 ➔ draft fourni dans [tp1_part0a.py](#)

Partie Q0 – question b

Construire une classe « Filtre de Kalman étendu »

Draft en Python: [ekf.py](#)

```
import numpy as np
class ExtendedKalmanFilter:
    def __init__(self, x0, P0):
        self.x = np.asarray(x0, dtype=float)
        self.P = np.asarray(P0, dtype=float)

    def predict(self, f, F_jacobian, Q):
        """
        Modèle:  $x_k = f_k(x_{k-1}) + w_k$  ;  $F_k = df_k/dx$ 
        """
        # TODO

    def update(self, y, h, H_jacobian, R):
        """
        Modèle de mesure:  $y_k = h_k(x_k) + v_k$  ;  $H_k =$ 
        """
        # TODO
        innovation = None
        S = None
        K = None
        return innovation, S, K
```

- La prédiction et la correction de l'estimation et de l'erreur sont placées dans les variables $x0$ et $P0$ de la classe
- On complètera la définition des 2 méthodes
- On testera la classe sur l'exemple fourni dans [tp1_part0b.py](#)
- **Qu'observe-on et pourquoi?**

converge puis
deverge

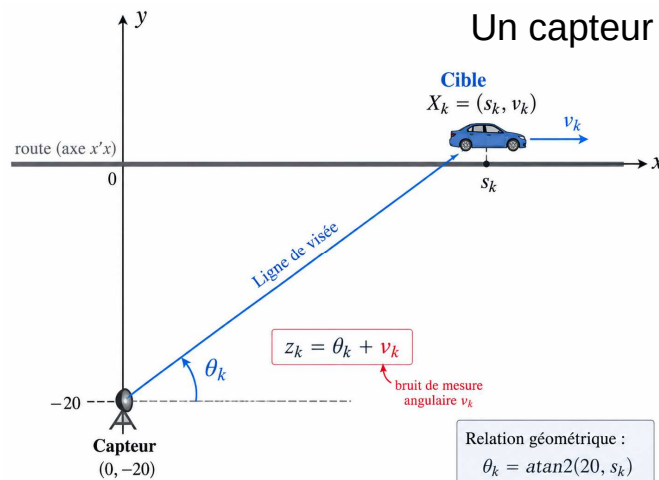
Question Q0b – description de l'exemple

Un objet se déplace sur la droite x ' x du plan.

Son état est décrit par les composantes de position et de vitesse

$$X_k = \begin{bmatrix} s_k \\ v_k \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} X_{k-1} + \eta_k \in \mathbb{R}^2 \text{ avec } \eta_k \sim \mathcal{N}\left(0, \begin{bmatrix} 0.0001 & 0 \\ 0 & 0.01 \end{bmatrix}\right)$$

Un capteur à la position $c = \begin{bmatrix} x_c \\ y_c \end{bmatrix} = \begin{bmatrix} 0 \\ -20 \end{bmatrix}$ effectue une mesure angulaire bruitée:



$$z_k = \theta_k + v_k \text{ avec } \theta_k = \text{atan2}(0 - y_k, s_k - x_k) = \text{atan}\left(\frac{0 - y_k}{s_k - x_k}\right)$$

$$v_k \sim \mathcal{N}(0, 1^{\circ 2})$$

Rappel:

$$d\theta = -\frac{y}{x^2 + y^2} dx + \frac{x}{x^2 + y^2} dy \text{ pour } \theta = \text{atan2}(y, x)$$

Initialisation: voir [tp1_part0b.py](#)

Plan

- Rappels
- TP1-Q0
- **TP1-Q1**
- TP1-Q2

Question Q1

Appliquer la classe KalmanFilter à un problème de localisation GNSS (ex. GPS) simplifié

→ On fait l'hypothèse de mesures GNSS indépendantes en choisissant Δt assez grand

Modèle d'évolution : $\Delta t = 1$

$$X_k = \begin{bmatrix} \text{Pos}_k \\ \text{Vel}_k \end{bmatrix} = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0_3 & I_3 \end{bmatrix} X_{k-1} + \eta_k \in \mathbb{R}^6 \quad \text{avec} \quad \eta_k \sim \mathcal{N}(0, Q)$$

Modèle d'observation :

$$Y_k = \text{Pos}_k + v_k = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} X_k + v_k \quad \text{avec} \quad v_k \sim \mathcal{N}(0, R)$$

- Implémenter l'exemple (simulation // prédiction, correction) avec le filtre de Kalman
- Simuler un trou de mesure (uniquement pour cette question), par exemple entre t=10 et 20 (exclus).
Qu'observe-t-on? Expliquer?
- Faire varier le bruit de dynamique du filtre (Q). Qu'observe-t-on? Expliquer?
- Faire varier le bruit de mesure du filtre (R). Qu'observe-t-on? Expliquer?

Initialisation: voir [tp1_part1.py](#)

Plan

- Rappels
- TP1-Q0
- TP1-Q1
- **TP1-Q2**

Question Q2

On utilisera et complètera le draft [tp1_part2_EKFLocalization.py](#)

Principe général:

- On a un robot qui se déplace. Son état contient une position et un cap $X_t = [x_t, y_t, \theta_t]^T$
- Le déplacement du robot est mesuré par odométrie: une valeur bruitée du déplacement **local** en position et en angle est obtenue:

$$U_t = [x_u, y_u, \theta_u]^T = u_t + \eta_k \quad \text{avec} \quad \eta_k \sim \mathcal{N}(0, Q)$$

u_t est un contrôle non aléatoire observé par odométrie de bruit η_k

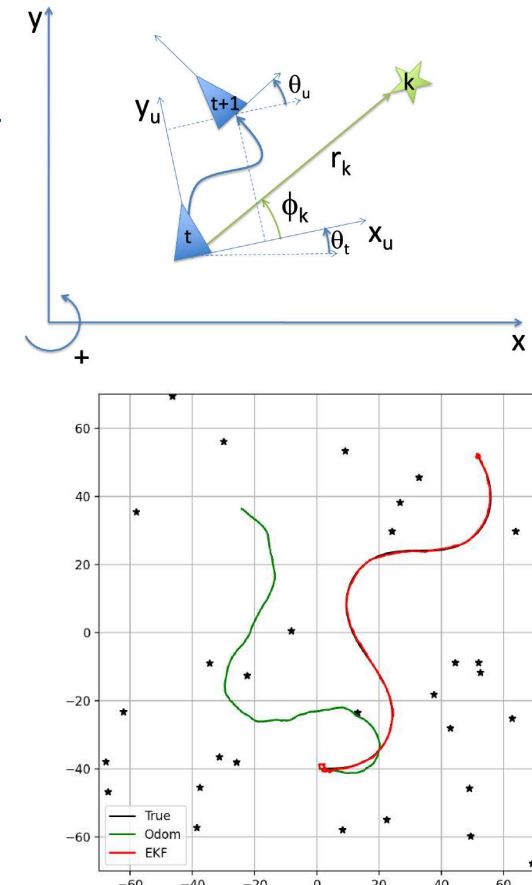
- L'état $X_{t+1|t} = f(X_t, U_t)$ s'obtient composition de l'état passé et du déplacement local:

$$X_{t+1} = f(X_t, U_t) = \begin{bmatrix} x_t + x_u \cos(\theta_t) - y_u \sin(\theta_t) \\ y_t + x_u \sin(\theta_t) + y_u \cos(\theta_t) \\ \theta_t + \theta_u \end{bmatrix}$$

Pour se localiser, le robot observera un amer $k = k_t$ (parmi un ensemble de points de repères) parfaitement identifiable, caractériser par la position $X_k = [x_k, y_k]^T$. L'observation produit une distance et un angle bruité:

$$Y_t^* = h^k(X_t^*) = \begin{bmatrix} \sqrt{(x_k - x_t)^2 + (y_k - y_t)^2} \\ \text{atan2}(\frac{y_k - y_t}{x_k - x_t}) - \theta_t \end{bmatrix} \quad \text{avec bruit additif} \quad v_k \sim \mathcal{N}(0, P_y)$$

Détails (précisions et questions) : voir le document [TP_EKF_Localization_2026.pdf](#)



Question Q2 – précision sur la prédiction de l'EKF

Ici le bruit de dynamique n'est pas additif, puisque :

$$X_{t+1} = f(X_t, U_t) \text{ avec } U_t = u_t + \eta_k \text{ et } \eta_k \sim \mathcal{N}(0, Q)$$

On utilise la différentielle respectivement par rapport à X et par rapport à U pour propager l'erreur:

Estimation prédiction : $\hat{x}_{k+1|k} = f_k(\hat{x}_k, \hat{u}_k)$ avec $\hat{u}_k$ donné par l'observation odométrique

Erreur prédiction : $P_{k+1|k} = A_{k+1}(\hat{x}_k, \hat{u}_k)P_{k-1}A_{k+1}^T(\hat{x}_k, \hat{u}_k) + B_{k+1}(\hat{x}_k, \hat{u}_k)QB_{k+1}^T(\hat{x}_k, \hat{u}_k)$ avec:
 $A_{k+1}(\hat{x}_k, \hat{u}_k) = D_x f(\hat{x}_k, \hat{u}_k)$ et $B_{k+1}(\hat{x}_k, \hat{u}_k) = D_u f(\hat{x}_k, \hat{u}_k)$

Détails (précisions et questions) : [TP_EKF_Localization_2026.pdf](#) et [tp1_part2_EKFLocalization.py](#)

- Initialisations, bruits, ...
- Le draft python gère lui-même le choix des amers
- Le draft contient des fonctions de visualisation, la boucle de simulation et d'ekf

→ Renseigner les jacobienes ; répondre aux questions